Code Running Guide

How to Run and Extend the Note 1 Python Examples

Lu-Xing Yang

Created from my research experience in optimal control, differential games, impulse and continuous–impulsive hybrid control, reinforcement learning, multi-agent learning, and cyber/network-security applications

Informed by publications in IEEE TIFS, TDSC, TSMC, TNSE, TCSS, and related journals

Part of the Network Control and Differential Games tutorial family

Purpose. Installation, smoke tests, training diagnostics, outputs, and troubleshooting

Companion to the cyber control and game-learning guide

July 7, 2026

Contents

1. What This Guide Covers

This guide is a standalone manual for the Python examples. It explains how to set up the environment, run smoke tests, run longer examples, interpret outputs, and extend the code to more complex cyber models.

2. Folder Structure

The repository is organized around one readable path from model to experiment:

```
.
  requirements.txt
  README.md
  README.md
  scripts/
    run_smoke_tests.sh
    generate_figures.py
    run_training_iterations.py
  src/
    cyber_dynamics.py
    sampled_continuous_impulse_env.py
    fbsm_malware_baseline.py
    ddqn_cyber_defense.py
    madrl_ctde_parameterized_game.py
    node_level_robustness.py
    evaluation_metrics.py
  artifacts/experiments/
    OUTPUT_PREVIEW.md
    training_summary.md
    policy_comparison_metrics.csv
    game_response_metrics.csv
    node_level_robustness_metrics.csv
  docs/assets/
    action_timing.png
    node_level_learning_advantage.png
```

3. Installation

Create a Python virtual environment and install dependencies:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
```

The examples require only NumPy, PyTorch, and optionally Matplotlib. CUDA is optional.

4. Quick Smoke Test

Run all scripts in quick mode:

```
bash scripts/run_smoke_tests.sh
```

A smoke test checks syntax and basic data flow. It does not train high-quality policies or prove game-theoretic convergence.

5. Normal Runs

5.1 Classical FBSM baseline

```
python src/fbsm_malware_baseline.py --max-iter 100
```

Expected output: iteration logs, peak infection, objective value, final state, and mean control.

5.2 DDQN defender

```
python src/ddqn_cyber_defense.py --episodes 300
```

Expected output: training return, validation return, and exploration rate. Improvement is stochastic; run multiple seeds.

5.3 MADRL attacker-defender game

```
python src/madrl_ctde_parameterized_game.py --episodes 200
```

Expected output: defender and attacker episode returns. This file is a compact CTDE teaching example. For research, extend it with MAPPO clipping, GAE, opponent pools, and cross-play evaluation.

5.4 Static figure generation

```
python scripts/generate_figures.py
```

Expected output: refreshed PNG figures under `docs/assets/`, including neural architecture, action timing, FBSM, sampled-flow rollout, and policy-comparison plots.

5.5 Longer training diagnostics

```
python scripts/run_training_iterations.py --episodes 180
```

Expected output:

- CSV histories under `artifacts/experiments/`;
- `OUTPUT_PREVIEW.md` and `training_summary.md`;
- the training-diagnostics, game-response, and node-level epidemic-robustness PNG figures.

6. How to Interpret Outputs

Output	Interpretation
Training return	Cumulative reward during learning. It is noisy and should not be the only metric.
Validation return	Performance of the current policy without exploration against a fixed scenario.
Peak infected	Maximum compromised population; useful cyber-risk metric.
Area under $I(t)$	Total compromise burden over time; often more informative than final infection.
Defender cost	Negative cumulative defender reward; lower is better when exposure is also controlled.
Impulse events	How often immediate jump actions, such as isolation, were used.
Game response matrix	Defender-policy performance against several attacker strategies.
Node-level epidemic robustness plot	Parameter-mismatch stress test: nominal FBSM open-loop schedule versus DDQN aggregate feedback on a stochastic graph where each node has a local S/I/R state.

7. Extending the Code

7.1 From simple malware to APT

Add a deception action and modify the infection rate during the affected decision interval:

$$\beta SI \quad \rightarrow \quad \beta(1-\chi\eta_k)VC, \qquad 0 \le \eta_k \le 1. \tag{1}$$

Here η_k is the deception intensity selected at action epoch t_k. Add actions such as deceive, isolate, and stealth. The provided environment already contains these mechanisms.

7.2 From compartment model to degree-based model

Replace S, I, R by degree-indexed compartments. For example,

$$x = [S_1, I_1, R_1, \ldots, S_K, I_K, R_K]. \tag{2}$$

The infection pressure should use the degree-weighted infected probability.

7.3 From scripted attacker to learned attacker

Start with `ddqn_cyber_defense.py`. Replace the scripted attacker with an actor network. Then use `madrl_ctde_parameterized_game.py` as the training template.

7.4 From one attacker to a response matrix

Use `evaluation_metrics.py` to add attacker policies, then rerun the training-diagnostics script. The output `game_response_metrics.csv` is the first step toward cross-play and exploitability analysis.

7.5 From tabular response to larger games

For larger games, keep the environment contract explicit: observe at action point t_k, apply jumps only when intended, integrate over $[t_k, t_{k+1})$, and report cyber metrics alongside rewards. A fixed interval is convenient, but a nonuniform action schedule is valid if it is recorded and used consistently. Then add opponent pools, multiple random seeds, and unilateral-deviation checks.

8. Troubleshooting

- If DDQN is unstable, reduce reward scale, reduce learning rate, enlarge replay buffer, and update target network less frequently.
- If MADRL cycles, freeze one agent for several updates, add opponent pools, and evaluate cross-play matrices.
- If a learned policy looks good in reward but bad in cyber metrics, inspect the reward formula and compare cumulative compromise, peak compromise, and defender cost.
- If an action label is unclear, rename it in `evaluation_metrics.py`; figures and CSVs will then inherit the clearer label.
- If outputs are missing, run the workflow in order: smoke tests, static figure generation, then longer training diagnostics.

9. Reproducibility Checklist

For every experiment, record:

- model equations and parameter values;
- random seed and initial states;
- action space and reward formula;
- ODE solver, action schedule, and solver step size;
- network architecture and optimizer;
- number of episodes or iterations;
- all baselines and attacker strategies used for comparison;
- output metrics: cumulative compromise, peak compromise, final compromise, defender cost, impulse count, and game-response table.